

Criando e Inserindo Elementos (createElement e appendChild)

1. O que acontece com o elemento antes do appendChild?

Ele fica no "limbo" da Memória RAM. O objeto existe como uma variável JavaScript completa — você pode mudar a cor, adicionar classes e textos — mas ele não tem uma posição física na árvore do DOM. Ele é um "nômade digital": está vivo no sistema, mas não tem endereço na página, por isso o usuário ainda não consegue vê-lo.

2. Por que o textContent é mais seguro que o innerHTML?

- O innerHTML é um interpretador de código. Se você inserir o texto "", o innerHTML vai tentar executar essa tag, o que permite ataques de injeção (XSS).
- Já o textContent é "literal". Ele trata tudo o que você escreve como texto puro. Se você passar o mesmo código malicioso para o textContent, o navegador apenas exibirá as palavras na tela, sem nunca transformá-las em um comando real. É a diferença entre ler um manual de instruções e realmente apertar os botões.

3. Código para criar o botão

Aqui estão as duas linhas clássicas:

JavaScript

```
const botao = document.createElement('button');  
botao.textContent = 'Enviar';
```

4. O comando em maiúsculas createElement('DIV') funciona?

Sim, funciona perfeitamente. No HTML (e no DOM para documentos HTML), os nomes das tags são case-insensitive (não diferenciam maiúsculas de minúsculas). O navegador entende que DIV, div e até div referem-se ao mesmo tipo de nó. No entanto, por convenção e boas práticas de legibilidade, quase todos os desenvolvedores utilizam letras minúsculas.

5. O appendChild duplica ou move o elemento?

Ele move. No DOM, um elemento é um objeto único. Ele não pode estar em dois lugares ao mesmo tempo, assim como você não pode estar em duas salas simultaneamente. Se você tem um botão no Contêiner A e faz um appendChild dele para o Contêiner B, o navegador o "desplugua" do primeiro e o "espeta" no segundo.

Curiosidade: Se você quisesse duplicar, teria que usar primeiro o método elemento.cloneNode(true) para criar uma cópia idêntica antes de anexar.

Removendo Elementos da Árvore (removeChild e remove)

1. Diferença Conceitual: removeChild vs remove

A diferença reside na perspectiva e na hierarquia:

- **removeChild(filho):** É uma visão centrada no pai. Você precisa primeiro localizar o contêiner superior para então ordenar que ele expulse um subordinado específico. É um método mais antigo (DOM Level 1).

- **remove()**: É uma visão centrada no próprio elemento. Ele tem autonomia para se retirar da árvore sem que o código precise saber quem é o seu pai. É um método moderno e muito mais conciso.

2. O Erro de "Paternidade"

Se você tentar executar `pai.removeChild(filho)`, mas o elemento "filho" não for um descendente direto daquele "pai", o console disparará um erro do tipo:

- **Uncaught NotFoundError**: Failed to execute 'removeChild' on 'Node': The node to be removed is not a child of this node.

O JavaScript protege a integridade da árvore, impedindo que um elemento "demit" alguém que não trabalha para ele.

3. Script para destruir o primeiro

JavaScript

```
const primeiroItem = document.querySelector('li');
if (primeiroItem) {
  primeiroItem.remove();
}
```

4. Guardar para usar depois?

Sim, perfeitamente possível! Quando você executa `.remove()`, o elemento sai da tela, mas continua vivo na variável enquanto o script estiver rodando.

Exemplo: Você pode remover um item de uma lista de "Tarefas Pendentes" e usar o `appendChild` para inseri-lo em uma lista de "Tarefas Concluídas". Ele não morre na memória até que você anule a variável ou o script termine.

5. O destino da Tabela (<tbody>)

No DOM, a relação é de aninhamento. Se você remove o `<tbody>`, todos os `<tr>` (linhas) e `<td>` (células) dentro dele são removidos da tela junto com ele.

Eles não ficam "órfãos" flutuando na página; eles permanecem conectados ao `<tbody>` no limbo da memória. Se você decidir anexar esse `<tbody>` em outra tabela depois, todos os seus filhos (linhas e células) reaparecerão intactos, pois a estrutura interna daquela "sub-árvore" foi preservada.

Se você apagar o "tronco" de uma árvore, todos os galhos e folhas que estão nele vão junto, certo? No DOM, a lógica é exatamente a mesma.

Renderização Dinâmica e Clean Code: A Função `renderList`

1. Por que limpar o container com `innerHTML = ''`?

Para evitar o efeito "**acumulador**". Se você chamar a função **renderList** toda vez que um novo dado chegar (ou quando o usuário filtrar uma busca), sem limpar o container, o JavaScript vai apenas pendurar os novos itens no final dos antigos. Limpar o container garante que a tela reflita exatamente o estado atual dos seus dados, e não uma pilha de restos de renderizações anteriores.

2. O que acontece se passar uma `String` no lugar de um `Array`?

O código vai quebrar e gerar um erro no console:

Uncaught TypeError: items.forEach is not a function

Isso ocorre porque o método `.forEach()` pertence exclusivamente ao protótipo de `Arrays`. Embora uma string possa ser iterada de outras formas, ela não possui esse método específico, interrompendo a execução do script.

3. Modificando a função com a classe CSS

Basta usar a propriedade `classList.add` antes de anexar o filho:

JavaScript

```
function renderList(dados, containerPai) {
  containerPai.innerHTML = "";

  dados.forEach(item => {
    const li = document.createElement('li');
    li.textContent = item;
    li.classList.add('list-item'); // Adiciona a classe CSS
    containerPai.appendChild(li);
  });
}
```

4. Como chamar a função na prática?

Considerando os dados fornecidos, a chamada seria:

JavaScript

```
const usuarios = ['Alice', 'Bob'];
const listaDOM = document.querySelector('#users');

renderList(usuarios, listaDOM);
```

5. Qual princípio de Engenharia de Software estamos aplicando?

Estamos aplicando o princípio DRY (Don't Repeat Yourself) — em português, "Não se repita".

Ao centralizar a lógica em uma função `renderList`, você remove a redundância. Além disso, aplicamos o conceito de Abstração, onde escondemos os detalhes complexos (criar elemento, configurar, anexar) atrás de um comando simples e reutilizável.

EXERCÍCIOS DE FIXAÇÃO

Exercício 1 — Notificação Simples

Neste exercício, utilizamos a tríade fundamental da criação dinâmica: criar o nó, configurar suas propriedades (classes e texto) e anexá-lo ao documento.

JavaScript

// 1. Criar o elemento

```
const alerta = document.createElement('div');
```

// 2. Configurar a personalidade

```
alerta.classList.add('alert-success');
```

```
alerta.textContent = 'Produto cadastrado com sucesso!';
```

// 3. Conectar ao DOM (adoção)

```
document.body.appendChild(alerta);
```

Exercício 2 — O Gerente Mandou Apagar

O método `.remove()` garante autonomia ao elemento para se destruir. O `if` é essencial para evitar o erro `TypeError` caso o alerta já não exista mais na tela.

JavaScript

```
const erroValidacao = document.querySelector('#erro-validacao');

if (erroValidacao) {
  erroValidacao.remove();
}
```

Exercício 3 — Categorias Dinâmicas

Criamos uma função que desacopla os dados da interface, garantindo que o seletor seja limpo antes de receber os novos `options`.

JavaScript

```
function renderizarCategorias(categorias) {
  const select = document.querySelector('#categoria');
  select.innerHTML = ''; // Limpeza de buffer

  categorias.forEach(cat => {
    const option = document.createElement('option');
    option.value = cat.id; // O valor oculto enviado ao servidor
    option.textContent = cat.nome; // O texto visível para o usuário
    select.appendChild(option);
  });
}
```

Exercício 4 — Refatoração de Acoplamento

Transformamos um código engessado em uma fábrica de rodapés genérica, eliminando o uso perigoso de `innerHTML +=`.

JavaScript

```
function renderizarRodape(container) {
  const div = document.createElement('div');
  div.classList.add('rodape');
  div.textContent = 'Total atualizado';

  // Agora a função anexa em qualquer container passado por parâmetro
  container.appendChild(div);
}
```

Exercício 5 — Adicionar e Sincronizar (Limpar Estoque)

A lógica guiada por dados (*Data-Driven*) dita que nunca esvaziamos o HTML primeiro. Esvaziamos o dado e mandamos a tela se atualizar.

JavaScript

```
function limparEstoque() {
  // 1. Manipula o Estado (Array)
  dadosProdutos = [];
```

```
// 2. Sincroniza a Tela  
renderizarTabela();  
}
```

Exercício 6 — Ações na Tabela Dinâmica

Construindo e aninhando múltiplos filhos dentro do mesmo pai (tdAcoes).

JavaScript

```
// Este trecho entra no loop forEach dentro de renderizarTabela()
```

```
const tdAcoes = document.createElement('td');
```

```
// Criação do botão Deletar (que já tínhamos)
```

```
const btnDeletar = document.createElement('button');
```

```
btnDeletar.textContent = 'Remover';
```

```
btnDeletar.className = 'btn btn-del';
```

```
// Criação do NOVO botão Editar
```

```
const btnEditar = document.createElement('button');
```

```
btnEditar.textContent = 'Editar';
```

```
btnEditar.className = 'btn btn-edit';
```

```
// Anexando os dois botões na mesma célula
```

```
tdAcoes.appendChild(btnEditar);
```

```
tdAcoes.appendChild(btnDeletar);
```

Exercício 7 — Fechamento por Escopo (Closure) no Botão

Graças ao escopo do `forEach`, a função anônima do botão "lembra" perfeitamente do objeto `produto` exato correspondente à sua linha no momento de sua criação.

JavaScript

```
// Logo abaixo da criação do btnEditar do exercício anterior:
```

```
btnEditar.onclick = () => {
```

```
  alert(`Editando o produto: ${produto.nome}`);
```

```
};
```

Exercício 8 — Dropdown Dependente

Combinando busca em objeto dinâmico e limpeza de DOM.

JavaScript

```
function renderizarSubcategorias(categoriaSelecionada) {
```

```
  const selectSub = document.querySelector('#subcategorias');
```

```
  selectSub.innerHTML = ""; // Zera a lista anterior
```

```
  // Busca a lista de subcategorias usando a chave recebida
```

```
  const itens = subcategorias[categoriaSelecionada];
```

```
  if (itens) {
```

```
    itens.forEach(subcat => {
```

```
      const option = document.createElement('option');
```

```
      option.textContent = subcat;
```

```
      selectSub.appendChild(option);
```

```
});  
}  
}
```

Exercício 9 — Cards de Portfólio (Visualização em Grade)

Avançando na hierarquia complexa: garantindo que `h3` e `p` sejam "irmãos", e o `div.card-produto` seja o pai de ambos.

JavaScript

```
function renderizarGaleria() {  
  const galeria = document.querySelector('#galeria');  
  galeria.innerHTML = ''; // Limpa a galeria  
  
  dadosProdutos.forEach(produto => {  
    // 1. Cria o Pai  
    const card = document.createElement('div');  
    card.classList.add('card-produto');  
  
    // 2. Cria os Filhos  
    const h3 = document.createElement('h3');  
    h3.textContent = produto.nome;  
  
    const p = document.createElement('p');  
    p.textContent = `R$ ${produto.preco.toFixed(2)}`;  
  
    // 3. Conecta a Família  
    card.appendChild(h3);  
    card.appendChild(p);  
  
    // 4. Injeta na Tela  
    galeria.appendChild(card);  
  });  
}
```

Exercício 10 — Depuração com Console (Code Review)

Do ponto de vista da Engenharia de Software, utilizar `innerHTML +=` em loops é uma péssima prática porque esse comando destrói todo o contêiner e o recria como texto puro a cada iteração. Isso causa o apagamento forçado de Event Listeners atrelados aos itens anteriores, gerando bugs ocultos. Além disso, aciona múltiplos ciclos custosos de *Reflow* e *Repaint* no navegador, estrangulando a performance, e abre portas para ataques de Cross-Site Scripting (XSS) se os dados não estiverem devidamente sanitizados. O padrão seguro e profissional é sempre manipular a árvore criando nós individualmente e usando `.appendChild`.

Após executar todos os exercícios o Sistema de Inventário estará completo. Abaixo, seguem algumas explicações dos exercícios aplicados ao código.

O que acontece neste Gabarito?

1. Dropdowns Interligados (Ex 3 e 8): Ao carregar a página, as funções `renderizarCategorias` e `renderizarSubcategorias` já disparam, preenchendo as caixas de seleção. Se você mudar a categoria, a subcategoria muda junto.
2. Alertas e Exclusões (Ex 1 e 2): Ao clicar em "Salvar", o aviso vermelho horrível é deletado via `.remove()`, e um alerta verde construído via `.createElement()` é anexado dinamicamente para confirmar o salvamento.
3. Escopo e Botões (Ex 6 e 7): A tabela agora possui um botão laranja de Editar. Graças ao conceito de *Closure* do JavaScript, ao clicar nele, o alerta exibirá exatamente o nome do produto correspondente àquela linha, sem se confundir.
4. Data-Driven Global (Ex 4 e 5): Se você clicar em "Limpar Estoque", perceba que limpamos o array e mandamos renderizar. Automaticamente, a tabela some e o "Rodapé" com o totalizador recalcula e some também.
5. DOM Complexo (Ex 9): O botão "Ver como Galeria" lê o mesmo Array (`dadosProdutos`) da tabela, mas forja uma estrutura de nós completamente diferente (`div > h3 + p + p`) e injeta no final da página.